МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ №3

з дисципліни **"Функціональне програмування"**

на тему **"НЕЗДАТНІ ДО ЗМІН СТРУКТУРИ ДАНИХ"**

Варіант 14. IoT-сенсори

Виконала:
Студентка групи ФЕП-23
Поведа Анастасія

Перевірив:
проф. Свелеба С. А.

Львів-2026

Мета: опанувати незмінні структури даних Python (`tuple` і `NamedTuple`, `frozenset`, `@dataclass(frozen=True, slots=True)`, `MappingProxyType`), перевести модель IoT-сенсорів з мутацій на повернення нових копій (pure updates) і виміряти, як це позначилося на тестах.

Теоретичні відомості:

Поряд із чистими функціями другою опорою функціонального стилю є незмінні дані. Якщо об'єкт не можна змінити після створення, вираз завжди дорівнює своєму значенню, спільне читання з кількох місць безпечне, а сам об'єкт можна хешувати — отже, використовувати як ключ словника, елемент множини чи аргумент `lru_cache`. У стандартній бібліотеці для цього є `tuple` і `NamedTuple` замість `list`, `frozenset` замість `set`, `@dataclass(frozen=True)` для сутностей предметної області (присвоєння полів заборонено, а «оновлення» виконує `dataclasses.replace`, що повертає копію) і `MappingProxyType` — представлення словника лише для читання.

Варто пам'ятати, що `MappingProxyType` є неглибоким: він захищає тільки верхній рівень і відображає зміни вихідного словника, тому джерело копіюють перед обгортанням. Щоб «заморозити» вкладені структури, застосовують рекурсивну утиліту, яка перетворює `dict`, `set` і `list` на `MappingProxyType`, `frozenset` і `tuple`. Платою за незмінність є створення нового об'єкта при кожному оновленні, однак це окупається відсутністю захисних копій і можливістю кешувати чисті обчислення.

Хід роботи:

1. За варіантом 14 (IoT-сенсори) спочатку зафіксувала мутабельну модель «до» у `mutable_model.py`: сенсор як `dict` зі списком показів, множиною міток і

вкладеним словником конфігурації; усі оновлення — in-place, статистика щоразу перераховується. Файл лишено для порівняння часу:

```python
def make_sensor_dict(id: int, kind: str) -> dict[str, Any]:
    """Сенсор як звичайний dict зі списком, множиною і вкладеним dict."""
    return {
        "id": id,
        "kind": kind,
        "readings": [],
        "labels": set(),
        "config": {},
        "enabled": True,
    }


def add_reading_inplace(sensor: dict[str, Any], ts: int, value: float) -> None:
    """Мутує список показів."""
    sensor["readings"].append({"ts": ts, "value": value})


def add_label_inplace(sensor: dict[str, Any], label: str) -> None:
    """Мутує множину міток."""
    sensor["labels"].add(label)


def stats_dict(sensor: dict[str, Any]) -> tuple[int, float, float, float]:
    """Статистика без кешу: щоразу перераховується."""
    values = [r["value"] for r in sensor["readings"]]
    if not values:
        return (0, 0.0, 0.0, 0.0)
    return (len(values), min(values), max(values), sum(values) / len(values))
```

2. У `model.py` описала незмінну модель: тип сенсора через `Literal`, показ `Reading` як `NamedTuple` (звичайний кортеж з іменованими полями `ts` і `value`) та сутність `Sensor` як `@dataclass(frozen=True, slots=True)`. Покази зберігаються у `tuple`, мітки у `frozenset`, конфігурація — як `Mapping` (read-only proxy). Методи

`add_reading`, `add_label`, `disable` повертають нові копії через `replace`, конкатенацію tuple та об'єднання frozenset:

```python
Kind = Literal["temperature", "humidity", "pressure"]


class Reading(NamedTuple):
    """Один показ сенсора: момент часу (с) та значення."""

    ts: int
    value: float


@dataclass(frozen=True, slots=True)
class Sensor:
    """Сенсор: покази (tuple), мітки (frozenset), config (read-only)."""

    id: int
    kind: Kind
    readings: tuple[Reading, ...]
    labels: frozenset[str]
    # mappingproxy не хешується → виключаємо з hash (у eq лишається)
    config: Mapping[str, float] = field(hash=False)
    enabled: bool = True

    def last_value(self) -> float | None:
        """Останній показ або None, якщо показів ще немає."""
        return self.readings[-1].value if self.readings else None

    def add_reading(self, reading: Reading) -> Sensor:
        """Новий сенсор із доданим показом."""
        return replace(self, readings=self.readings + (reading,))

    def add_label(self, label: str) -> Sensor:
        """Новий сенсор із доданою міткою."""
        return replace(self, labels=self.labels | frozenset({label}))

    def disable(self) -> Sensor:
        """Новий сенсор, вимкнений."""
        return replace(self, enabled=False)

```

3. Додала `make_config`, яка обгортає у `MappingProxyType` копію словника (щоб ніхто зовні не тримав посилання на джерело), і зручний конструктор `make_sensor`:

```python
def make_config(**kwargs: float) -> Mapping[str, float]:
    """Read-only view над КОПІЄЮ словника, щоб ніхто не тримав посилання на оригінал."""
    return MappingProxyType(dict(kwargs))


def make_sensor(
    id: int,
    kind: Kind,
    readings: tuple[Reading, ...] = (),
    labels: frozenset[str] = frozenset(),
    **config: float,
) -> Sensor:
    """Зручний конструктор сенсора з read-only конфігурацією."""
    return Sensor(id, kind, readings, labels, make_config(**config))
```

4. У `core.py` реалізувала чисті бізнес-функції: результат статистики `Stats` як `NamedTuple`, кешований через `@lru_cache` розрахунок `stats` (можливий лише тому, що `Sensor` хешований), перевірку `is_out_of_range` за межами з конфігурації та pure-оновлення `add_reading` (показ поза межами додає мітку

`alert`), `add_label`, `with_config` (нова read-only конфігурація з копії та змін), `recalculate`:

```python
class Stats(NamedTuple):
    """Статистика показів: кількість, мінімум, максимум, середнє."""

    n: int
    low: float
    high: float
    mean: float


@lru_cache(maxsize=1024)
def stats(sensor: Sensor) -> Stats:
    """Статистика за всіма показами сенсора. Кешується."""
    values = [r.value for r in sensor.readings]
    if not values:
        return Stats(0, 0.0, 0.0, 0.0)
    return Stats(len(values), min(values), max(values), sum(values) / len(values))


def is_out_of_range(sensor: Sensor, value: float) -> bool:
    """Чи виходить значення за межі min/max із конфігурації (якщо вони задані)."""
    low = sensor.config.get("min", float("-inf"))
    high = sensor.config.get("max", float("inf"))
    return not low <= value <= high


def add_reading(sensor: Sensor, ts: int, value: float) -> Sensor:
    """Pure update: новий сенсор із доданим показом; поза межами → мітка alert."""
    updated = sensor.add_reading(Reading(ts, value))
    return updated.add_label("alert") if is_out_of_range(sensor, value) else updated


def add_label(sensor: Sensor, label: str) -> Sensor:
    """Pure update: новий сенсор із доданою міткою."""
    return sensor.add_label(label)


def with_config(sensor: Sensor, **updates: float) -> Sensor:
    """Pure update: новий сенсор із новою read-only конфігурацією (копія + зміни)."""
    merged = {**sensor.config, **updates}
    return Sensor(
        sensor.id,
        sensor.kind,
        sensor.readings,
        sensor.labels,
        MappingProxyType(merged),
        sensor.enabled,
    )


def recalculate(sensor: Sensor) -> tuple[Sensor, Stats]:
    """Повертає той самий сенсор і його статистику (кешовану)."""
    return sensor, stats(sensor)

```

5. Додала `index_by_sensor`, де сенсор використовується як ключ словника, та утиліту `freeze` для глибокого заморожування вкладених структур. Поле `config`

(`mappingproxy`) не має `__hash__`, тому в моделі воно виключене з хешу через `field(hash=False)`, але бере участь у порівнянні:

```python
def index_by_sensor(sensors: tuple[Sensor, ...]) -> Mapping[Sensor, Stats]:
    """Сенсор як ключ словника (хешованість frozen dataclass) → статистика."""
    return MappingProxyType({s: stats(s) for s in sensors})


def freeze(obj: Any) -> Any:
    """Глибоке заморожування: dict → MappingProxyType, set → frozenset, list → tuple."""
    if isinstance(obj, Mapping):
        return MappingProxyType({k: freeze(v) for k, v in obj.items()})
    if isinstance(obj, (set, frozenset)):
        return frozenset(freeze(x) for x in obj)
    if isinstance(obj, (list, tuple)):
        return tuple(freeze(x) for x in obj)
    return obj
```

6. Демонстраційний скрипт `main.py` будує ланцюжок pure-оновлень (два покази, один із них поза межами, мітка та нова конфігурація), показує, що початковий сенсор не змінився, друкує стан кешу та перевіряє read-only поведінку конфігурації:

```python
def main() -> None:
    """Створює сенсор, виконує три pure-оновлення і показує, що оригінал не змінився."""
    s0 = make_sensor(
        1, "temperature", (Reading(0, 21.5),), frozenset({"room"}), min=18.0, max=25.0
    )
    s1 = add_reading(s0, 60, 22.5)
    s2 = add_reading(s1, 120, 27.0)
    s3 = with_config(add_label(s2, "checked"), max=30.0)

    for name, s in [("s0", s0), ("s1", s1), ("s2", s2), ("s3", s3)]:
        st = stats(s)
        print(
            f"{name}: readings={len(s.readings)} labels={sorted(s.labels)} "
            f"config={dict(s.config)} last={s.last_value()} mean={st.mean:.2f}"
        )
    print(
        "s0 is s3:",
        s0 is s3,
        "| s0 unchanged:",
        s0.readings == (Reading(0, 21.5),) and s0.labels == frozenset({"room"}),
    )
    stats(s3)
    print("cache:", stats.cache_info())
    index = index_by_sensor((s0, s2))
    print("as dict keys:", len(index), "sensors ->", [v.n for v in index.values()])
    try:
        s0.config["max"] = 99.0  # type: ignore[index]
    except TypeError as e:
        print("config read-only:", e)


if __name__ == "__main__":
    main()
```

7. Написала тести у `tests/test_immutability.py`: відсутність мутації після додавання показу та мітки, автоматична мітка `alert`, `FrozenInstanceError` при

присвоєнні поля, показ є кортежем без можливості змінити поле, `TypeError` при записі у proxy, копіювання джерела `make_config`, нова мапа у `with_config`:

```python
def test_add_reading_returns_new_sensor(sensor: Sensor) -> None:
    before = (sensor.readings, sensor.labels, dict(sensor.config))
    new = add_reading(sensor, 120, 23.0)
    assert (sensor.readings, sensor.labels, dict(sensor.config)) == before
    assert new is not sensor
    assert len(new.readings) == 3
    assert new.last_value() == 23.0 and sensor.last_value() == 22.5


def test_out_of_range_reading_adds_alert_label(sensor: Sensor) -> None:
    new = add_reading(sensor, 120, 27.0)
    assert "alert" in new.labels and "alert" not in sensor.labels
    assert is_out_of_range(sensor, 27.0) and not is_out_of_range(sensor, 20.0)


def test_add_label_returns_new_sensor(sensor: Sensor) -> None:
    new = add_label(sensor, "checked")
    assert sensor.labels == frozenset({"room"})
    assert new.labels == frozenset({"room", "checked"})
    assert new.disable().enabled is False and sensor.enabled is True


def test_frozen_fields_cannot_be_assigned(sensor: Sensor) -> None:
    with pytest.raises(FrozenInstanceError):
        sensor.kind = "humidity"  # type: ignore[misc]


def test_reading_is_tuple(sensor: Sensor) -> None:
    reading = sensor.readings[0]
    assert isinstance(reading, tuple) and reading == (0, 21.5)
    with pytest.raises(AttributeError):
        reading.value = 0.0  # type: ignore[misc]


def test_config_is_read_only(sensor: Sensor) -> None:
    with pytest.raises(TypeError):
        sensor.config["max"] = 99.0  # type: ignore[index]
    assert sensor.config["max"] == 25.0


def test_make_config_copies_source() -> None:
    raw = {"min": 1.0}
    config = make_config(**raw)
    raw["min"] = 2.0
    assert config["min"] == 1.0


def test_with_config_creates_new_mapping(sensor: Sensor) -> None:
    new = with_config(sensor, max=30.0, period=60.0)
    assert dict(new.config) == {"min": 18.0, "max": 30.0, "period": 60.0}
    assert dict(sensor.config) == {"min": 18.0, "max": 25.0}

```

8. Додала тести на хешованість (сенсори у `set` і як ключі словника), на кеш

`stats` через `cache_info` та на глибоке заморожування:

```python
def test_hashability_and_dict_key(sensor: Sensor) -> None:
    other = add_label(sensor, "checked")
    assert len({sensor, other, add_label(sensor, "checked")}) == 2
    index = index_by_sensor((sensor, other))
    assert index[sensor].mean == pytest.approx(22.0)
    assert index[other] == index[sensor]
    assert set(index) == {sensor, other}


def test_stats_and_cache(sensor: Sensor) -> None:
    stats.cache_clear()
    assert stats(sensor) == (2, 21.5, 22.5, 22.0)
    stats(sensor)
    info = stats.cache_info()
    assert info.hits == 1 and info.misses == 1
    assert stats(add_reading(sensor, 120, 27.0)).high == 27.0
    assert stats(make_sensor(2, "humidity")) == (0, 0.0, 0.0, 0.0)


def test_freeze_deep() -> None:
    frozen = freeze({"a": [1, {2, 3}], "b": {"c": [4]}})
    with pytest.raises(TypeError):
        frozen["a"] = 0
    assert frozen["a"] == (1, frozenset({2, 3}))
    with pytest.raises(TypeError):
        frozen["b"]["c"] = 0
    assert frozen["b"]["c"] == (4,)

```

9. Property-тести на Hypothesis генерують випадкові сенсори й покази та перевіряють, що `add_reading` чистий, а рівні сенсори мають однаковий хеш і однакову кешовану статистику:

```python
readings_st = st.tuples(
    st.integers(min_value=0, max_value=10**6),
    st.floats(min_value=-50, max_value=150, allow_nan=False),
).map(lambda t: Reading(*t))
sensors_st = st.builds(
    make_sensor,
    st.integers(min_value=1),
    st.sampled_from(["temperature", "humidity", "pressure"]),
    st.tuples(readings_st, readings_st).map(tuple),
    st.frozensets(st.text(min_size=1, max_size=6)),
)


@given(sensors_st, readings_st)
def test_property_add_reading_pure(s: Sensor, r: Reading) -> None:
    before = (s.readings, s.labels, dict(s.config))
    new = s.add_reading(r)
    assert (s.readings, s.labels, dict(s.config)) == before
    assert new.readings == s.readings + (r,)
    assert new.last_value() == r.value


@given(sensors_st)
def test_property_hash_consistent_with_eq(s: Sensor) -> None:
    twin = Sensor(s.id, s.kind, s.readings, s.labels, s.config, s.enabled)
    assert twin == s and hash(twin) == hash(s)
    assert stats(s) == stats(twin)
```

10. Для вимірювання ефекту у `tests/test_performance.py` два тести з однаковим навантаженням (2000 додавань показу і 2000 розрахунків

статистики) на мутабельній моделі «до» та незмінній «після»:

```python
N = 2_000


def test_mutable_before() -> None:
    s = make_sensor_dict(1, "temperature")
    for i in range(N):
        add_reading_inplace(s, i, 20.0 + i % 7)
    add_label_inplace(s, "checked")
    total = sum(stats_dict(s)[3] for _ in range(N))
    assert total > 0


def test_immutable_after(sensor: Sensor) -> None:
    s = sensor
    for i in range(N):
        s = add_reading(s, i, 20.0 + i % 7)
    s = add_label(s, "checked")
    stats.cache_clear()
    total = sum(stats(s).mean for _ in range(N))
    assert total > 0
    assert stats.cache_info().hits == N - 1
```

11. Запустила демонстрацію та порівняння часу `pytest --durations=5`: незмінна модель із кешованим розрахунком виконує те саме навантаження приблизно у 15–20 разів швидше (0.01 с проти 0.18 с), бо 1999 із 2000 звернень до статистики — cache hit:

```
C:\Users\maksi\Desktop\labs_functional_for_nastia\lab03> python main.py
s0: readings=1 labels=['room'] config={'min': 18.0, 'max': 25.0} last=21.5 mean=21.50
s1: readings=2 labels=['room'] config={'min': 18.0, 'max': 25.0} last=22.5 mean=22.00
s2: readings=3 labels=['alert', 'room'] config={'min': 18.0, 'max': 25.0} last=27.0 mean=23.67
s3: readings=3 labels=['alert', 'checked', 'room'] config={'min': 18.0, 'max': 30.0} last=27.0 mean=23.67
s0 is s3: False | s0 unchanged: True
cache: CacheInfo(hits=1, misses=4, maxsize=1024, currsize=4)
as dict keys: 2 sensors -> [1, 3]
config read-only: 'mappingproxy' object does not support item assignment
```

```
C:\Users\maksi\Desktop\labs_functional_for_nastia\lab03> python -m pytest -q --durations=5
...............                                                          [100%]
============================ slowest 5 durations =============================
0.19s call     lab03/tests/test_performance.py::test_mutable_before
0.11s call     lab03/tests/test_immutability.py::test_property_add_reading_pure
0.10s call     lab03/tests/test_immutability.py::test_property_hash_consistent_with_eq
0.01s call     lab03/tests/test_performance.py::test_immutable_after

(1 durations < 0.005s hidden.  Use -vv to show these durations.)
15 passed in 0.68s
```

12. Перевірила проєкт автоматичними інструментами: усі 15 тестів pytest проходять, `mypy --strict` не знаходить помилок типів, `ruff` та `black` не мають

зауважень до стилю:

```
C:\Users\maksi\Desktop\labs_functional_for_nastia\lab03> python -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\maksi\AppData\Local\Python\pythoncore-3.14-64\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\maksi\Desktop\labs_functional_for_nastia
configfile: pyproject.toml
plugins: hypothesis-6.170.0, platformdirs-4.12.3
collecting ... collected 15 items

tests\test_immutability.py::test_add_reading_returns_new_sensor PASSED   [  6%]
tests\test_immutability.py::test_out_of_range_reading_adds_alert_label PASSED [ 13%]
tests\test_immutability.py::test_add_label_returns_new_sensor PASSED     [ 20%]
tests\test_immutability.py::test_frozen_fields_cannot_be_assigned PASSED [ 26%]
tests\test_immutability.py::test_reading_is_tuple PASSED                 [ 33%]
tests\test_immutability.py::test_config_is_read_only PASSED              [ 40%]
tests\test_immutability.py::test_make_config_copies_source PASSED        [ 46%]
tests\test_immutability.py::test_with_config_creates_new_mapping PASSED  [ 53%]
tests\test_immutability.py::test_hashability_and_dict_key PASSED         [ 60%]
tests\test_immutability.py::test_stats_and_cache PASSED                  [ 66%]
tests\test_immutability.py::test_freeze_deep PASSED                      [ 73%]
tests\test_immutability.py::test_property_add_reading_pure PASSED        [ 80%]
tests\test_immutability.py::test_property_hash_consistent_with_eq PASSED [ 86%]
tests\test_performance.py::test_mutable_before PASSED                    [ 93%]
tests\test_performance.py::test_immutable_after PASSED                   [100%]

============================== 15 passed in 0.68s ==============================

C:\Users\maksi\Desktop\labs_functional_for_nastia\lab03> python -m mypy .
Success: no issues found in 7 source files

C:\Users\maksi\Desktop\labs_functional_for_nastia\lab03> python -m ruff check .
All checks passed!

C:\Users\maksi\Desktop\labs_functional_for_nastia\lab03> python -m black --check .
All done! ✨ 🍰 ✨
7 files would be left unchanged.
```

Висновок: Під час виконання лабораторної роботи я перевела мутабельну модель IoT-сенсорів (варіант 14) на незмінну: показ `Reading` як `NamedTuple`, сенсор `Sensor` як frozen dataclass зі slots, покази у `tuple`, мітки у `frozenset`, конфігурація у `MappingProxyType` над копією джерела. Усі оновлення реалізовано як чисті функції, що повертають нові копії через `dataclasses.replace`; хешованість сенсора дозволила кешувати розрахунок статистики через `lru_cache` і використовувати сенсор як ключ словника. Тести підтвердили відсутність мутацій і read-only поведінку, а порівняння часу показало, що незмінна модель не лише не сповільнила код, а завдяки кешуванню значно його прискорила; накладні витрати — копіювання колекцій при кожному оновленні та захист лише верхнього рівня у proxy, який за потреби доповнює `freeze`.